

RAG 高级检索：混合检索、Rerank 与 GraphRAG

语言策略：code (Java / Python / TypeScript)

TL;DR

- 基础 RAG 的主要瓶颈不是“有没有检索”，而是召回不准、上下文杂乱、问题表达不稳定。
- 生产上优先按顺序做三件事：**混合检索 + RRF 融合 → Rerank 精排 → 查询改写**；GraphRAG 只用于多跳关系问题。
- 每条检索链路都要可评测、可回放：命中率、MRR、引用准确率必须进入回归门禁。

1. 从基础 RAG 到高级检索

阶段	做法	解决什么
基础 RAG	单路向量召回 top-k	语义相似问题
混合检索	向量 + BM25 + 元数据过滤	专有名词、编号、时间范围
融合	RRF 或加权融合多路结果	多路排序尺度不统一
精排	Cross-Encoder Rerank	前 50 条里选最相关的 5 条
查询改写	改写、扩展、HyDE	口语化、多义词、查询过短
图检索	GraphRAG	多跳关系与全局总结

2. Chunk 策略

策略	适用场景	注意
固定长度	快速起步	语义会被切断
段落/标题	结构化文档	需要保留层级元数据
句子窗口	精读任务	检索用小句，送入模型带上下文窗口
父子文档	命中后扩上下文	父块要单独存储
表格/代码独立切	技术文档	不要按行切开

原则：切分结构要与检索结构一致；每次切分策略变更都要重跑评测集。

3. 查询处理

- 查询改写：把“上次那个报错怎么修”改写成“订单服务 OOM 报错修复方法”；
- 查询扩展：生成同义词或多个子查询；
- HyDE：先用 LLM 生成假设答案，再用答案向量去召回；
- 多轮对话：先做指代消解，把“它”替换为具体对象，再检索；
- 元数据过滤：时间、租户、文档类型先过滤，再做语义召回。

4. 混合检索与 RRF

BM25 擅长专有名词和编号，向量检索擅长语义。两路分数不可直接相加，常用 Reciprocal Rank Fusion：

```
1 RRF_score(doc) = sum( 1 / (60 + rank_i(doc)) )
```

实现要点：先分别取每路 top-100，再融合取 top-20，最后交给 Rerank 取 top-5。

5. Rerank

- Cross-Encoder 比 Bi-Encoder 精度高、延迟高，所以只对候选集精排；

- 离线评测用 MRR、nDCG 和引用准确率，不要只看“感觉更相关”；
- 候选集通常 20-50 条，精排后送入上下文 3-8 条；
- 工具选择、模型切换必须过回归门禁。

6. GraphRAG 什么时候上

问题类型	普通 RAG	GraphRAG
单文档问答	够用	不需要
专有名词/编号	混合检索	不需要
多跳关系	容易断链	更适合
全局总结	容易偏	社区摘要更有结构

GraphRAG 成本高、更新难，先确认任务真的需要实体关系和社区摘要，再引入。

7. 检索评测

- 指标：命中率、MRR、nDCG、上下文引用准确率；
- 评测集：真实问题 + 期望文档 ID + 禁止返回文档；
- 每次改动 Chunk、Embedding、融合参数、Rerank 模型都必须回归；
- 线上 Bad Case 回流：检索失败、引用错误要进入下一轮评测集。

8. TypeScript 版检索融合实现

```
1 interface ScoredDoc {
2   docId: string;
3   score: number;
4   source: "vector" | "bm25" | "graph";
5 }
6
7 function rrfFusion(rankedLists: ScoredDoc[], k = 60, topN = 20):
8   ScoredDoc[] {
9   const scores = new Map<string, number>();
10  const sources = new Map<string, ScoredDoc["source"]>();
11
12  for (const list of rankedLists) {
13    for (let i = 0; i < list.length; i++) {
14      const doc = list[i];
15      scores.set(doc.docId, (scores.get(doc.docId) ?? 0) + 1 / (k +
16        i + 1));
17      if (!sources.has(doc.docId)) sources.set(doc.docId, doc.source
18    );
19    }
20  }
21
22  return [...scores.entries()]
23    .sort((a, b) => b[1] - a[1])
24    .slice(0, topN)
25    .map(([docId, score]) => ({ docId, score, source: sources.get(
26      docId! })));
27 }
```

9. 延伸阅读

- [Agent 评测体系](#)
- [AI 应用系统设计](#)